

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks

Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Bekbolatuly Yernar
Student Identification Number	230109011
Date & Laboratory Session	September 17, 2026
Workstation / Laptop Make & Model	ASUS ROG Strix G713RM_G713RM
Processor SKU / SoC Model	AMD Ryzen 9 6900HX with Radeon Graphics
Core Topology (Physical / Logical)	Physical Cores: 8 Logical SMT Threads: 16
Cache Hierarchy Capacity	L1: 576 KB L2: 4 MB L3: 16 MB
Host OS & Python Runtime	Windows 11 / CPython 3.13 (64-bit)

Note: Cache hierarchy capacity (L1/L2/L3) sourced from AMD official specifications for the Ryzen 9 6900HX SoC, as this is not directly exposed by a single OS command on this platform.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Total Weight: 25 Points

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency
p = 1 (Baseline)	4.8572	1.00x	1.00x	100.0%
p = 2	2.5227	1.93x	2.00x	96.5%
p = 4	1.4542	3.34x	4.00x	83.5%
p = 8	0.9501	5.11x	8.00x	63.9%
p = 12	0.8391	5.79x	12.00x	48.3%
p = 16	1.0830	4.48x	16.00x	28.0%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology. (8 Points)

The observed speedup curve peaks at $p^* = 12$ (5.79x, 48.3% efficiency), after which speedup drops to 4.48x at $p = 16$. My processor is an AMD Ryzen 9 6900HX with 8 physical cores and 16 logical SMT threads (2-way SMT per core, no heterogeneous P/E-core split as found on recent Intel hybrid designs). Physical core saturation actually begins at $p = 8$, where efficiency has already fallen to 63.9% from 96.5% at $p = 2$ — this shows that once every physical core is occupied, additional processes must start sharing execution resources through SMT rather than gaining dedicated hardware. The peak at $p = 12$ rather than $p = 8$ reflects that the OS scheduler can still partially load-balance work across the 8 physical cores when only some SMT threads are populated, yielding a small residual gain. At $p = 16$, however, every SMT thread on every physical core is active simultaneously, so each thread pair on a given core is now forced to fully contend for the shared front-end decoder, execution ports, and L1/L2 cache. This is the classic SMT degradation pattern: on integer CPU-bound work like this benchmark, two logical threads competing for the same physical resources yield no net throughput gain and can even regress, compounded further here by the fork/spawn overhead of launching 16 separate OS processes and additional OS context-switching.

[Q1.2] Using your empirical measurements for T(1) and T(4), calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation. (9 Points)

Using $T(1) = 4.8572s$ and $T(4) = 1.4542s$, so $S(4) = T(1)/T(4) = 3.34x$.

$$S(4) = 1 / [(1-P) + P/4]$$

$$3.34 = 1 / [(1-P) + 0.25P]$$

$$(1-P) + 0.25P = 1 / 3.34 = 0.2994$$

$$1 - 0.75P = 0.2994$$

$$0.75P = 0.7006$$

$$P \approx 0.9341 \text{ (93.4\%)}$$

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1-P)$ using your calculated P from Q1.2. Explain what this limit implies for a 128-core server node. (8 Points)

$$S_{\max} = 1 / (1 - P) = 1 / (1 - 0.9341) = 1 / 0.0659 \approx 15.17x.$$

This means that even with an infinite number of cores, this specific workload can never speed up by more than roughly 15x, because approximately 6.6% of the total execution time is fundamentally non-parallelizable (process serialization, spawn/fork latency, and inter-process communication overhead). If the department purchased a 128-core server node to run this exact workload, cores beyond roughly the 16th would provide essentially no additional benefit — the money spent on the remaining 112+ cores would be largely wasted capacity sitting idle relative to this task. In fact, my own empirical data at $p = 16$ (4.48x) already underperforms even the theoretical asymptotic ceiling of 15.17x, since real-world OS scheduling overhead and SMT contention degrade performance faster than Amdahl's idealized model predicts.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Total Weight: 25 Points

Memory Layout	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median [s]	Slowdown Multiplier
Adjacent Indices (False Sharing)	1.8070	1.7712	1.7264	1.7712	1.00x [Baseline]
Padded Indices (Cache-Aligned)	1.6492	1.6754	1.7467	1.6754	—

Overall Slowdown Penalty Multiplier (median-based): $1.7712 / 1.6754 \approx \mathbf{1.06x}$

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees no shared 64-byte L1 cache line. (8 Points)

In CPython, a reference stored inside a list is a pointer to a PyObject, which occupies **8 bytes** on a 64-bit architecture (this is the size of the machine pointer itself, distinct from the larger boxed integer object it points to). The standard cache line size on x86-64 architectures, including AMD Zen 3+ cores used in the Ryzen 9 6900HX, is **64 bytes**. With a stride of 16 indices, the byte offset between two consecutively accessed slots is $16 \times 8 \text{ bytes} = \mathbf{128 \text{ bytes}}$. Since $128 > 64$, this guarantees that each accessed slot lands in its own distinct 64-byte cache line (128 bytes spans exactly two full cache lines), so at least one full 'buffer' line always separates the two active worker indices. Formally: if the array base address is A , element i sits at address $A + i \times 8$. The distance between `padded_idx = 0` and `padded_idx = 16` is 128 bytes = 2×64 bytes, an exact multiple of the cache line size that exceeds it — hence the two threads can never physically share a cache line.

[Q2.2] Trace the MESI state transitions for the shared cache line when Core 0 increments index 0 and Core 1 subsequently increments index 1. (9 Points)

In a fully parallel, compiled-language scenario, the transitions would proceed as follows: (1) Core 0 increments index 0 → the cache line transitions to **Modified** in Core 0's L1, and any other cached copies are broadcast an Invalidate signal, moving to **Invalid**. (2) Core 1 then wants to increment index 1, which resides in the *same* line → since the line is Modified in Core 0, Core 1 must issue a Bus Read/RFO (Read-For-Ownership) request. Core 0 flushes its modified copy back, the line briefly becomes **Shared**, then immediately **Invalid** on Core 0 once Core 1 acquires it exclusively and transitions it to **Modified** locally. (3) This Modified → Invalidate → Shared → Modified cycle repeats on every loop iteration for both threads, producing a continuous cache-line 'ping-pong' between cores — the root cause of pipeline stalls. **Important caveat from my empirical data:** the measured effect in CPython was quite weak (slowdown $\approx 1.06x$, and in one of the three trials padded was even slightly slower than adjacent). This is because CPython's Global Interpreter Lock (GIL) serializes bytecode execution, so true simultaneous memory access from different physical cores at the exact moment of increment rarely occurs: while one thread holds the GIL and executes the increment, the others are physically stalled. The classic MESI ping-pong seen in compiled C/C++ code with genuine parallelism is therefore heavily dampened here — the small measurable effect mainly comes from OS context switches placing threads on different physical cores between GIL acquisitions, not from truly concurrent writes to a shared line.

[Q2.3] Describe two industry-standard mechanisms to eliminate false sharing without manual padding. (8 Points)

1. C++11 alignas(64) — an alignment attribute instructing the compiler to place a variable/struct starting strictly on a 64-byte boundary, guaranteeing it occupies its own cache line and never shares one with neighboring struct fields. **2. Java @Contended** (from `sun.misc.Contended`, JEP 142) — the JVM automatically inserts padding around an annotated field during object memory layout, isolating it in its own cache line without the developer manually computing byte offsets. Both mechanisms shift the responsibility for alignment from the developer to the compiler/VM, reducing the risk of manual errors and keeping the source code clean, unlike manual array padding which wastes memory (as seen in the Task 2 benchmark, where the padded array is 16x larger than the number of elements actually used).

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Total Weight: 25 Points

Counter Architecture	Target Sum	Actual Sum	Defect Count	Time [s]	Contention Overhead
Unsafe Accumulator	2,000,000	2,000,000	0	0.2356	1.00x [Baseline]
Locked Accumulator	2,000,000	2,000,000	0	0.7729	3.28x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence that caused lost updates. (7 Points)

In my specific run, the Unsafe Accumulator produced exactly **2,000,000** — i.e., no data corruption occurred. This is a valid and important result worth explaining rather than a failed experiment: in naive C/C++ implementations with true parallel execution, a race condition on `val += 1` almost always loses updates, because the operation decomposes into three discrete machine steps: **LOAD** (read the current value of `val` from memory into a register), **ADD** (increment the register by 1), and **STORE** (write the result back to memory). If Thread A completes **LOAD** and **ADD** but has not yet executed **STORE** when Thread B also performs **LOAD** (reading the same stale value), both threads eventually write results derived from the same original value, and one increment is silently lost. In CPython, this race did not manifest in this particular run because of the Global Interpreter Lock (GIL): the GIL allows only one thread to execute bytecode at any instant, and although a context switch can technically still occur mid-sequence (interrupting **LOAD-ADD-STORE**), with a relatively modest workload ($2,000,000 / 4 = 500,000$ operations per thread) the probability of a switch landing exactly inside the vulnerable window was low enough that no corruption surfaced in this run. This illustrates a key distinction from compiled languages: the GIL does not guarantee atomicity of `+=`, but it statistically suppresses the frequency of visible race conditions compared to genuinely parallel execution across physical cores.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator. Record your measured execution time and calculate the speedup factor over `LockedCounter`. (11 Points)

Implementation approach (Map-Reduce pattern): Each of the 4 threads accumulates into its own private `local_sum` variable, living entirely in that thread's stack frame and inaccessible to any other thread. Once its loop completes, the thread performs a single write of its final value into a dedicated slot of a results array (`results[thread_id]`) — a write with no race, since that slot is owned exclusively by that thread. After all threads join, a single reduction step, `sum(results)`, combines the four private results into the final total. No lock is required at any point during the hot loop, because there is no shared mutable state to protect.

```
import threading, time
TOTAL_OPS = 2_000_000; NUM_THREADS = 4

def bench_lockless():
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    results = [0] * NUM_THREADS # one private slot per thread
    def work(thread_id):
        local_sum = 0 # private, on this thread's stack
        for _ in range(ops_per_thread):
            local_sum += 1
        results[thread_id] = local_sum # single write, no race
    threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    total = sum(results) # reduction stage
    return total, time.perf_counter() - start
```

Measured results: LockedCounter = 0.7437s; Lockless Thread-Local Accumulator = 0.1329s.

Speedup Factor = $0.7437 / 0.1329 \approx 5.60x$ (requirement was $\geq 2.0x$ — exceeded by nearly 2.8x). Both requirements are satisfied: correctness (2,000,000 / 2,000,000 deterministically every run, since there is no shared counter to race on) and performance (5.60x faster than LockedCounter). The speedup comes from completely eliminating lock acquisition/release on each of 2,000,000 iterations — replaced by just 4 slot writes plus one reduction pass.

[Q3.3] Why is eliminating shared mutable state fundamentally superior to optimizing lock primitives in high-throughput concurrent systems? (7 Points)

Eliminating shared mutable state is fundamentally superior because it attacks the root of the problem rather than its symptom. Any mutex — even an adaptive spinlock or a read-write lock — still requires cross-core synchronization over the interconnect: atomic compare-and-swap operations used to acquire a lock themselves generate cache-coherence traffic (analogous to the MESI invalidation traced in Task 2), and under contention this can escalate to futex system calls, kernel-level context switches, and thread suspension. The faster and 'smarter' the lock, the smaller this overhead becomes, but it never reaches zero, because the logical requirement to serialize access never disappears. Thread-local buffers remove the need for any cross-core coordination during the most performance-critical part of the computation (the hot loop) — each core operates exclusively on its own private memory, which resides comfortably in its own L1 cache with no conflict misses and no invalidation broadcasts across the interconnect. Synchronization is deferred to a single, short reduction phase at the end, whose cost scales with the number of threads $O(n)$ rather than with the number of operations $O(\text{total_ops})$ — which is exactly why the lockless version outperforms LockedCounter by such a wide margin in my benchmark: synchronization cost is paid once at the end, not on every one of 2,000,000 individual increments.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Total Weight: 25 Points

Worker Allocation	Compute-Bound Time [s]	Compute Scaling (Tw/T1)	Memory-Bound Time [s]	Memory Degradation (Tw/T1)
w = 1 (Baseline)	1.8844	1.00x	0.9511	1.00x
w = 2	1.8629	0.99x	0.8971	0.94x
w = 4	2.0374	1.08x	1.3437	1.41x

[Q4.1] Analyze the scaling divergence between the two workloads: why does 4 parallel compute tasks complete in roughly the same time as 1, while 4 parallel memory tasks spike dramatically? (9 Points)

The compute-bound task performs 25,000,000 arithmetic operations entirely in CPU registers, with zero main memory traffic in the hot loop. Each worker uses its own independent execution pipeline (ALU/FPU) on its own core, so 4 parallel processes do not physically contend for any shared resource — they are as isolated from one another as if running on 4 separate machines. This is why execution time at $w = 4$ (2.0374s) is nearly identical to $w = 1$ (1.8844s), with only a small ~8% increase attributable to process-spawn overhead from multiprocessing.Pool. The memory-bound task, by contrast, allocates roughly 400 MB per worker and must physically read/write that data through the shared DRAM bus and the single memory controller common to the entire processor. Unlike execution units, the memory bus and its bandwidth are a **shared resource** across all cores. At $w = 1$, a single process can use the full available bandwidth. At $w = 4$, four processes simultaneously attempt to push data through that same physical bus, creating a queue of requests at the memory controller — the total data volume requiring transfer has quadrupled, but the bus bandwidth has not increased at all. The result is the observed 41% time increase (1.41x), since adding workers does not expand the physical capacity of the bus, it only lengthens the queue of pending requests.

[Q4.2] Investigate your host machine's physical memory specifications and calculate the theoretical peak memory bandwidth. Explain how memory bus width limits multi-core scalability. (8 Points)

My system has 2 Micron DDR5 memory modules, 8 GB each (16 GB total), running at **4800 MT/s** — this is a **DDR5-4800 Dual-Channel** configuration, since two identical modules are populated in separate slots. **Theoretical peak bandwidth calculation:** a single DDR5 channel transfers 8 bytes per cycle (64-bit data bus). Single-channel bandwidth = $4800 \text{ MT/s} \times 8 \text{ bytes} = 38,400 \text{ MB/s} \approx \mathbf{38.4 \text{ GB/s}}$. In a Dual-Channel configuration this bandwidth doubles: $38.4 \times 2 \approx \mathbf{76.8 \text{ GB/s}}$ theoretical peak for this system. Memory bus width directly limits multi-core scalability: regardless of how many logical cores (16 in my case) attempt to transfer data simultaneously, aggregate traffic through the memory controller cannot exceed roughly 76.8 GB/s. Once several cores saturate this bandwidth at once — exactly what happens in Task 4 at $w = 4$ — additional cores no longer increase throughput, they simply compete for a fixed resource, which is precisely why the observed performance degrades instead of improving.

[Q4.3] For a memory-bandwidth-saturated ML embedding pipeline, would CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup? Justify your architectural choice. (8 Points)

If the embedding pipeline for high-dimensional feature vectors is memory-bandwidth saturated — meaning the bottleneck is data movement through the memory bus rather than raw computation — then optimizing with **CUDA GPU kernels** will yield significantly greater speedup than CPU OpenMP pragmas. My Task 4 results directly demonstrate why: the core problem in memory-bound workloads is a limited, shared DRAM bus ($\approx 76.8 \text{ GB/s}$ theoretical peak on my DDR5-4800 Dual-Channel system). OpenMP simply adds more CPU threads/cores that continue competing for that same limited bus — my measurements show this only worsens the request queue (a 1.41x slowdown going from 1 to 4 workers), rather than resolving the underlying constraint. Modern GPUs use a fundamentally different memory architecture — GDDR6X or HBM VRAM with bandwidth in the 500–3000+ GB/s range (an order of magnitude beyond DDR5 systems) — combined with thousands of parallel CUDA cores designed specifically to hide memory latency through massive parallelism (warp scheduling). For high-dimensional embedding operations (fundamentally linear-algebra work: matrix multiplications, dot products), the GPU architecturally eliminates exactly the bottleneck I measured empirically in Task 4 — limited DRAM bandwidth, not insufficient compute units.

BONUS TASK: THREADING VS. MULTIPROCESSING SCALING & THE GIL AUTOPSY

Workers (T)	multiprocessing.Pool Time [s]	threading.Thread Time [s]
1	4.3197	4.1762
2	2.3802	4.3423
4	1.4362	4.7182
8	0.9568	4.1579
16	0.8605	4.1491
32	1.1113	4.1479

Total work per test: 50,000,000 units, split evenly across T workers. See attached bonus_task_plot.png for the plotted curve.

Deliverable 2 — The Inflection Cliff

multiprocessing.Pool reaches its minimum execution time at **T = 16** (0.8605s), which matches exactly the number of logical threads on my AMD Ryzen 9 6900HX (8 physical cores + SMT = 16). At **T = 32**, time increases again to 1.1113s — a classic case of **oversubscription**: requesting twice as many processes as available logical execution slots forces the OS to continuously preempt and context-switch 32 processes across only 16 logical threads, adding pure overhead with no corresponding gain. threading.Thread shows no meaningful inflection at all — execution time stays in a narrow 4.15–4.72s band regardless of T, which is itself the central finding of this experiment.

Deliverable 3 — The Architectural Autopsy

Why does threading remain flat/worse while multiprocessing scales (the GIL): CPython implements a Global Interpreter Lock (GIL) — a global mutex that allows only **one thread** to execute Python bytecode at any given instant, regardless of how many physical cores are available. My data confirms this directly: threading execution time barely changes from T = 1 (4.1762s) to T = 32 (4.1479s) — adding threads produces no performance gain whatsoever on this CPU-bound task, because true parallel execution never occurs in CPython threads: at any moment only one thread is active while the rest wait to acquire the GIL. The small bump at T = 4 (4.7182s) is pure GIL hand-off overhead with no compensating benefit. multiprocessing.Pool, in contrast, spawns fully independent OS processes, each with its own interpreter and its own separate GIL — so they genuinely execute in parallel across different physical cores with no contention for a shared lock. This is why multiprocessing follows the classic Amdahl-style curve: near-linear speedup up to T = 8 (physical cores), a small further gain up to T = 16 via SMT, and degradation at T = 32 due to oversubscription.

CPU topology: AMD Ryzen 9 6900HX — **8 physical cores, 16 logical threads** (2-way SMT per core).

Point of physical saturation and the T > physical-cores performance drop: Physical core saturation is reached at T = 8 (0.9568s), where every process already occupies a dedicated physical core. Between T = 8 and T = 16, time still improves slightly (down to 0.8605s), as the remaining 8 logical SMT threads still provide a modest benefit by filling idle execution ports during stall cycles within each core. But once T = 32 exceeds even the number of logical threads (16) by a factor of two, the OS is forced to run more processes than there are hardware execution slots to host them simultaneously — the scheduler must **preempt** processes, saving and restoring their context (registers, stack state, TLB entries) on every switch. This adds pure overhead unrelated to useful computation, and explains the measured rise in time from 0.8605s to 1.1113s empirically observed on my machine.